

Chapitre 7

Décidabilité

Enjeu :

- Comprendre les limites du calcul automatique
- Quels sont les problèmes pour lesquels on est capable de produire un algorithme qui y répond

Il est nécessaire de définir :

- la notion de problème
- la notion d'algorithme

Définition - Problème de décision

Un problème de décision est défini par la donnée d'une fonction booléenne $f : D \rightarrow \mathbb{B}$. Le problème de décision associé à f est spécifié par :

- Entrée : $x \in D$
- sortie $f(x)$

Remarque : D sera toujours dénombrable car un ordinateur ne peut manipuler que des objets dénombrables.

Définition - Algorithme (modèle)

- Machine de Turing
- λ -calcul (*Church*)
- fonctions μ -récursives (*Herbrand-Godel*)

Ces 3 modèles coïncident

Dans le cadre du programme :

Définition - Programme

Un programme est un programme écrit en C ou en OCaml (ou tout autre langage Turing complet) qui compile (mais qui ne s'arrête pas forcément) avec une mémoire infinie.

La mémoire infinie sert à ne pas considérer un problème comme indécidable faute de mémoire

1 Existence de problèmes indécidables

L'ensemble des programmes est dénombrable (c'est un sous-ensemble des chaînes de caractères) → procédé diagonal de cantor (cf loann).

En utilisant un procédé diagonal on peut construire une fonction $f : e_i \rightarrow \neg p_i(e_i)$.
Ainsi, aucun p_i ne calcule f qui n'est pas calculable

Définition - calculabilité

Soit $f : D \rightarrow \mathbb{B}$, on dit que f est **calculable** ou de manière équivalente que le problème associé est **décidable** s'il existe un programme qui prend en entrée $x \in D$ et **s'arrête** en renvoyant la valeur $f(x)$.

On a vu qu'il existait une fonction f non calculable.
Étant donné que l'ensemble des fonctions de D dans $\mathbb{B}$ (avec D fixé strictement dénombrable) n'est pas dénombrable, il existe forcément une infinité de problèmes définis sur D non calculables.

Proposition

Si D est fini alors toute fonction booléenne sur D est calculable.

DEMO : demo1

1.1 Problème de l'arrêt (HP)

Soit p un programme ayant un prototype $qqchose \rightarrow bool$.
On note

- $\langle p \rangle$ de type *string* : c'est la description du programme
- x une entrée du programme de type *qqchose*
- $\langle x \rangle$ de type *string* sa description

Définition - Le problème de l'arrêt

Entrée : $\langle p \rangle$ et $\langle x \rangle$

Sortie :

- **true** si l'exécution de p sur x s'arrête
- **false** sinon

Théorème

Le problème arrêt est indécidable

DEMO : demo2

2 Décidabilité

Définition - semi-décidabilité

Un problème associé à $f : d \rightarrow \mathbb{B}$ est dit **semi-décidable** $\Leftrightarrow$ il existe un programme qui prend $x \in D$ en entrée et renvoie :

- si $f(x)$ alors le programme s'arrête et renvoie **true**
- sinon alors il s'arrête et renvoie **false** ou ne s'arrête pas.

Exemple : Le problème d'arrêt est semi-décidable. (Exemple 1)

Théorème - machine universelle

Il existe un programme OCaml (resp. C) spécifié par :

$$u : \text{string} \rightarrow \text{string} \rightarrow \text{bool}$$

tel que pour tout programme p et toute entrée x :

$$u(\langle p \rangle, \langle x \rangle) = p(x)$$

si p s'arrête sur x et ne s'arrête pas sinon

Ce programme s'appelle **machine universelle**.

Remarque : La théorie de la calculabilité repose sur le fait que tous les objets manipulés sont en définitive de même nature : *string* (ou même un élément de $\{0;1\}^*$).

2.1 Exemples de problèmes indécidables

- Problème 1

Entrée : la description d'un programme $\langle p \rangle$ avec $p : \mathbb{N} \rightarrow \mathbb{B}$.

Sortie : **true**, si $\forall x$ premier, p s'arrête sur x $p(x) = \text{true}$

DEMO : demo3

- Problème 2 : Arrêt_univ

Entrée : $\langle p \rangle$ la description d'un programme $p : \text{string} \rightarrow \text{bool}$

Sortie :

— **true** $\Leftrightarrow p$ s'arrête sur chaque chaîne de caractère passée en entrée.

DEMO : demo4

Définition - Réduction d'un algorithme

Soient A associé à $f_1 : D_1 \rightarrow \mathbb{B}$, B associé à $f_2 : D_2 \rightarrow \mathbb{B}$.

On dit que A se réduit à B , on note $A \leq B$ (ou $A \leq_m B$) qui traduit l'idée que si on a un algo pour B alors on peut en déduire un pour A de la manière suivante :

$$\exists g : D_1 \rightarrow D_2 \text{ calculable tq } \forall x \in D_1, f_1(x) = \text{true} \Leftrightarrow f_2(g(x)) = \text{true}$$

Théorème

Soient f_1, f_2 tq $f_1 \leq f_2$.

Alors

$$f_1 \text{ calculable} \Rightarrow f_2 \text{ calculable}$$

et donc (contraposée)

$$f_2 \text{ non-calculable} \Rightarrow f_1 \text{ non-calculable}$$

DEMO :

- Problème 3 : Equiv

Entrée : 2 programmes $p_1, p_2 : \text{string} \rightarrow \text{bool}$

Sortie : p_1 et p_2 ont exactement le même comportement sur chaque entrée.

DEMO :

Définition - problème complémentaire

Soit P un problème de décision associé à $f : D \rightarrow \mathbb{B}$, on définit coP associé à $\tilde{f}$ le problème complémentaire à P avec

$$\tilde{f} : \begin{array}{l|l} D & \longrightarrow \mathbb{B} \\ x & \longmapsto \neg f(x) \end{array}$$

Exemple :

Proposition

P décidable $\Leftrightarrow coP$ décidable

Théorème

P et coP semi-décidables $\Rightarrow P$ décidable

DEMO :

2.2 Problème de Post

Problème de Post

Entrée : un ensemble fini de dominos

Sortie : true $\Leftrightarrow$ il existe une manière de positionner les dominos pour obtenir le même mot en haut et en bas (en pouvant les utiliser les plusieurs fois).

EXEMPLE.

Insérer exemple.

Proposition

Le problème de Post est indécidable

DÉMONSTRATION.